A COMPLETE STEP-BY-STEP LEARNING ROADMAP

FROM ZERO TO PRO IN JAVASCRIPT

The structured path to mastering Playwright test automation

12 PHASES | 60+ DAY PLAN | JS > TS > PLAYWRIGHT

Built for QA and automation engineers moving from Selenium and Java into modern JavaScript testing.

SWIPE TO BEGIN >

Shashank | AI QA Engineer
linkedin.com/in/dearshashank

READ THIS FIRST

Why this roadmap is different

Most JavaScript courses teach you to build websites. You are not building websites. You are building a test automation framework.

So the syllabus is deliberately biased

- **Front-loaded** with what Playwright uses every single day: async, objects, modules and classes.
- **De-emphasises** DOM manipulation, animation and browser APIs you will rarely write yourself.
- **Ordered by dependency** , not by difficulty. Each phase unlocks the next one.
- **Ends in TypeScript** , because the entire Playwright ecosystem is written in it.

How to use it

- Phases 0 to 4 are **non-negotiable** before you touch Playwright.
- You can start writing real tests after **Phase 5** .
- Learn Phases 6 to 10 **in parallel** with actual automation work. It sticks far better than pure study.

> Day estimates assume 1.5 to 2 focused hours a day. Move faster or slower as you like. The **sequence** matters more than the schedule.

THE BIG PICTURE

The 12 phases at a glance

00	Environment & Mental Model	1-2 days
01	Language Fundamentals	3-5 days
02	Functions & Scope	3-4 days
03	Objects, Arrays & Strings	4-6 days
04	**Asynchronous JavaScript**	5-7 days
05	Object-Oriented JavaScript	4-5 days
06	Modules & Project Structure	2-3 days
07	Error Handling & Debugging	2 days
08	Node.js Essentials	3-4 days
09	TypeScript Fundamentals	5-7 days
10	Web Fundamentals for Locators	2-3 days
11	Bridging Into Playwright	ongoing

> Phase 04 is highlighted for a reason. If your async is weak, your tests will be flaky and you will not know why.

PHASE 00 | 1-2 DAYS

Environment & mental model

Before syntax, understand where your code actually runs. Playwright is a Node program, not a browser script.

What to cover

- What JavaScript really is: **engine, runtime, single-threaded execution**
- **Node.js vs browser JavaScript** , and why that distinction matters for Playwright
- Installing Node, and the difference between **npm** and **npx**
- **package.json** : dependencies vs devDependencies, and npm scripts
- Running a .js file, and using the Node REPL
- **CommonJS (require) vs ES Modules (import / export)** . Know both. Playwright uses the ESM and TypeScript style.

> Outcome: you can create a project from scratch, install a package and run a script without copying commands blindly.

PHASE 01 | 3-5 DAYS

Language fundamentals

The grammar. Unglamorous, unavoidable, and the source of most silent bugs in test code.

What to cover

- **let, const and var** , and why you will almost never use var
- Primitive types: string, number, boolean, null, undefined, symbol, bigint
- **Type coercion** , == versus ===, and the truthy / falsy table
- **Template literals** with backticks, used constantly for dynamic URLs and selectors
- Operators: arithmetic, logical, ternary, **nullish coalescing (??)** and **optional chaining (?.)**
- Conditionals: if / else and switch
- Loops: for, **for...of** , for...in, while, plus break and continue

> Common trap: using == inside a helper and having the string zero quietly equal the number zero.

PHASE 02 | 3-4 DAYS

Functions & scope

Every Playwright test you write is a function handed to a test runner. Understand what that means.

What to cover

- Function declarations versus function expressions
- **Arrow functions** , and how they differ. They have no own **this** .
- Parameters: default values and **rest parameters (...args)**
- Return values and early returns
- Scope: global, function and block
- **Closures** , the backbone of custom fixtures and helper factories
- **Higher-order functions** , and callbacks
- The **this** keyword and its binding rules, which mostly matter inside classes

Closures feel abstract until you write your first custom fixture. Then they click permanently.

PHASE 03A | PART 1 OF 2

Objects: the shape of your data

Config, test data, API payloads, fixtures. Almost everything you touch is an object.

What to cover

- Object literals and nested objects
- Dot notation versus **bracket notation** , and computed property names
- Adding, updating and deleting properties
- **Object.keys(), Object.values(), Object.entries(), Object.assign(), Object.freeze()**
- **Spread operator** for merging and cloning config objects
- **Shallow versus deep copy** , and structuredClone()
- **Destructuring** , the single most visible pattern in Playwright:

```
test('login works', async ({ page, request }) => {
  const { username, password } = testData.admin;
});
```

PHASE 03B | PART 2 OF 2

Arrays & strings

Handling multiple locators, building data-driven tests, and parsing whatever the UI hands back.

Arrays

- Creation, indexing, length
- Mutating: push, pop, shift, unshift, splice
- Non-mutating: slice, concat, join
- **map, filter, reduce, find, findIndex, some, every, forEach, sort, flat**
- Array destructuring, spread, and **Array.from()** for converting locator collections

Strings & regex

- split, trim, includes, startsWith, endsWith, replace, replaceAll, slice, toLowerCase, padStart
- **Regular expression basics** . Playwright locators accept regex directly, for example getByText(/login/i)

> some() and every() map beautifully onto assertions made across a whole list of elements.

PHASE 04A | THE MOST IMPORTANT PHASE

Asynchronous JavaScript

Almost every Playwright API returns a Promise. This is the phase that separates a reliable suite from a flaky one.

Core mechanics

- Synchronous versus asynchronous execution
- **The event loop** : call stack, callback queue and microtask queue
- Callbacks, and how callback hell happens
- **Promises** : new Promise, resolve, reject, then, catch, finally
- Promise chaining, and why it was largely replaced
- **async and await** , and what await actually pauses
- Error handling in async code with try / catch / finally

> If you master only one phase deeply, make it this one. Everything downstream depends on it.

Promises in practice

Combinators, pitfalls, and the habits that keep a suite green.

Combinators

- **Promise.all()** : run in parallel and fail fast. Used to wait on a navigation and a click together.
- **Promise.allSettled()** : run everything and collect every outcome
- **Promise.race()** and **Promise.any()** : first to settle, first to succeed

Pitfalls to internalise

- **Forgetting await** and silently asserting on a pending Promise
- setTimeout and setInterval, and why hard waits are an anti-pattern
- Unhandled promise rejections, and where they surface
- Writing your own async utility, such as a **retry wrapper**

> A missing await rarely throws. It just makes your test pass for the wrong reason.

PHASE 05 | 4-5 DAYS

Object-oriented JavaScript

This phase maps directly onto the Page Object Model. Coming from Java, most of it will feel familiar.

What to cover

- **class** syntax and the constructor
- Instance properties, methods, and **this** inside classes
- **Inheritance** with extends and super. Your BasePage lives here.
- Static methods and properties, getters and setters
- **Private fields** using the hash prefix
- **Method chaining** by returning this, for fluent page objects
- **Composition versus inheritance** : when a helper class beats another BasePage
- Prototypes and the prototype chain. Conceptual understanding is enough.

> Deep inheritance trees are the most common design mistake in Page Object frameworks. Prefer composition.

PHASE 06 | 2-3 DAYS

Modules & project structure

A framework is judged by how quickly a new joiner can find things inside it.

What to cover

- **Named exports versus default exports**
- import and export syntax, and re-exports through barrel files
- **Circular dependency** pitfalls, and how to spot them
- Path aliases configured through tsconfig.json

A structure that scales

- **pages/** for page objects and reusable components
- **tests/** for specs grouped by feature, not by page
- **fixtures/** for custom fixtures and setup
- **utils/** for pure helpers with no Playwright imports
- **data/** for test data, payloads and constants

PHASE 07 | 2 DAYS

Error handling & debugging

A failing test that cannot explain itself costs more time than the bug it found.

What to cover

- The **Error** object, and throw
- try / catch / finally in both synchronous and async code
- **Custom error classes** , for example a TimeoutError that extends Error
- Reading a **stack trace** properly instead of scrolling past it
- console.log, **console.table** and console.dir
- Debugging with **VS Code breakpoints** and the Node inspector
- Understanding unhandled promise rejections

> Custom errors turn expected true, got false into cart badge never updated after adding SKU-441.

PHASE 08 | 3-4 DAYS

Node.js essentials for automation

The plumbing around your tests: data, config, artefacts and API setup.

What to cover

- **fs** for reading test data and writing JSON, screenshots and reports
- **path** : join, resolve, and __dirname with its ESM equivalent
- **process.env** and environment variables through dotenv
- **process.argv** for command line arguments
- child_process basics for setup scripts
- **JSON.parse() and JSON.stringify()**
- Dates for test data generation, using Date or a library such as dayjs
- **fetch()** from Node, for API setup and teardown alongside UI tests

> Seeding state through the API instead of the UI is the single biggest speed win in most suites.

PHASE 09 | 5-7 DAYS

TypeScript fundamentals

Playwright docs, examples and the wider ecosystem all default to TypeScript. With a Java background this will feel natural.

What to cover

- Why TypeScript, and tsconfig.json basics
- Basic types, arrays and tuples
- **interface versus type** , and union, intersection and literal types
- Optional and readonly properties
- Typing function parameters and return values
- **Generics** , needed to read Playwright's own type definitions
- enum, type inference, and type assertions using **as**
- **unknown versus any versus never**
- Utility types: **Partial, Pick, Omit, Record**
- Typing your page objects and custom fixtures

PHASE 10 | 2-3 DAYS

Web fundamentals for locators

You do not need to write front-end code. You absolutely need to read it.

What to cover

- HTML structure, tags, attributes, and **data attributes**
- **CSS selectors** : class, id, attribute, descendant, combinators, nth-child
- **XPath basics** , the fallback for when CSS cannot reach
- The DOM tree, and **Shadow DOM** . Playwright pierces it, but know what it is.
- **Accessibility roles, labels and ARIA** , the foundation of getByRole and getByLabel
- Browser DevTools: the Elements, Console and Network tabs
- iframes, browser events, and storage such as localStorage and cookies

> Prefer role and label based locators over raw CSS. They break less often and they document intent.

PHASE 11

Bridging into Playwright

Once the earlier phases are solid, these map one to one.

JAVASCRIPT CONCEPT	WHERE IT SHOWS UP IN PLAYWRIGHT
async / await	Every single action and assertion
Object destructuring	async ({ page, request }) => { }
Classes & inheritance	The Page Object Model
Modules	Importing page objects and fixtures
Closures	Custom fixtures via test.extend()
Array methods	locator.all() and data-driven loops
Promise.all()	Concurrent waits and parallel setup
Custom error classes	Meaningful failure messages
Generics & interfaces	Typed fixtures and test data
Regular expressions	Text and URL matching in locators
fs and JSON	Test data and stored auth state

BEFORE YOU START

Two pieces of hard-won advice

1. Sequence beats speed

- Phases **0 to 4** before you open Playwright. No exceptions.
- Start writing **real tests after Phase 5** . A working page object beats another tutorial.
- Fold Phases **6 to 10** into real work rather than studying them cold.

2. The trap for Selenium engineers

- The biggest habit people carry over from Java and Selenium is treating Playwright's **auto-waiting** as if it were an explicit wait.
- That misunderstanding is an **async JavaScript gap** , not a Playwright gap.
- Which is exactly why Phase 04 deserves more time than it looks like it needs.

> Learn the language first and the tool becomes obvious. Learn the tool first and every failure looks like magic.

THAT IS THE FULL ROADMAP

Now go and write the tests.

Save this so you can work through it phase by phase.

Repost it if it would help someone in your network.

Tell me which phase you are on. I reply to every comment.

Shashank

AI QA Engineer

linkedin.com/in/dearshashank